

Simulate Datasets

Setup

```
library(furrr)
library(tidyverse)
library(data.table)
library(arrow)
library(rms)
library(here)
```

Load data

```
df_ypprev <- read_parquet(here("data", "hosp_long_weeks_two_lags.parquet")) |>
  mutate(
    tx = factor(tx),
    ecog_fstcnt = factor(ecog_fstcnt, ordered = FALSE),
    yprev = factor(yprev, levels = 1:4, ordered = FALSE),
    ypprev = factor(ypprev, levels = 1:4, ordered = FALSE),
    y = ordered(y)
  ) |>
  select(pat_id, tx, y, yprev, ypprev, week, ecog_fstcnt, diagnosis, age, gender,
    albumin, c_reactive_protein)

d <- df_ypprev
```

Model fitting

We fit a model without a treatment parameter. We will randomly assign a treatment effect during the simulation of the dataset. We will use the same for model for HA, but simulate a treatment effect during the simulation of the dataset.

```
fit_h0 <- orm(
  y ~ rcs(week, 5) + yprev + ypprev + week %ia% yprev + week %ia% ypprev +
  ecog_fstcnt + gender + diagnosis + rcs(albumin, 3) + rcs(c_reactive_protein, 3) +
  rcs(age, 3),
  data = d
)

fit_h0
```

Logistic (Proportional Odds) Ordinal Regression Model

```
orm(formula = y ~ rcs(week, 5) + yprev + ypprev + week %ia% yprev +
  week %ia% ypprev + ecog_fstcnt + gender + diagnosis + rcs(albumin,
  3) + rcs(c_reactive_protein, 3) + rcs(age, 3), data = d)
```

Frequencies of Responses

1	2	3	4	5
5827	292	37	409	41

Discrim.		Model Likelihood		Discrimination	Rank
Indexes		Ratio Test		Indexes	
Obs	6606	LR chi2	1448.09	R2	0.318 rho
0.368					
ESS	2070.1	d.f.	29	R2(29,6606)	0.193 Dxy
0.643					
Distinct Y	5	Pr(> chi2)	<0.0001	R2(29,2070.1)	
0.496					
Median Y	1	Score chi2	2290.97	Pr(Y>=median)-0.5	
0.412					
max deriv	4e-12	Pr(> chi2)			
<0.0001					

	Coef	S.E.	Wald Z	Pr(> Z)
y>=2	-3.1427	0.8676	-3.62	0.0003
y>=3	-3.8855	0.8689	-4.47	<0.0001
y>=4	-4.0080	0.8692	-4.61	<0.0001
y>=5	-6.9595	0.8858	-7.86	<0.0001
week	0.0333	0.0407	0.82	0.4137
week'	-0.5812	0.2888	-2.01	0.0441
week''	1.4789	0.7727	1.91	0.0556
week'''	-1.6249	0.9726	-1.67	0.0948
yprev=2	1.6249	0.2237	7.26	<0.0001
yprev=3	0.5472	0.9297	0.59	0.5561
yprev=4	3.4679	0.2901	11.95	<0.0001
ypprev=2	0.0671	0.2390	0.28	0.7787
ypprev=3	2.1397	0.8883	2.41	0.0160
ypprev=4	-1.1123	0.3588	-3.10	0.0019
week * yprev=2	0.0636	0.0193	3.29	0.0010
week * yprev=3	0.0080	0.0776	0.10	0.9181
week * yprev=4	0.0490	0.0219	2.24	0.0254
week * ypprev=2	-0.0084	0.0229	-0.37	0.7134
week * ypprev=3	-0.1929	0.1002	-1.92	0.0542
week * ypprev=4	0.0334	0.0261	1.28	0.2004
ecog_fstcnt=2	0.3598	0.1181	3.05	0.0023
ecog_fstcnt=3	0.5056	0.1973	2.56	0.0104
gender=1	0.2254	0.0996	2.26	0.0237
diagnosis=2	-0.1526	0.1291	-1.18	0.2372
diagnosis=3	-0.2884	0.2312	-1.25	0.2123
diagnosis=4	-1.0559	0.1941	-5.44	<0.0001
diagnosis=5	0.0165	0.1311	0.13	0.9000
albumin	-0.0002	0.0212	-0.01	0.9915
albumin'	0.0212	0.0220	0.97	0.3334
c_reactive_protein	0.0145	0.0071	2.04	0.0412
c_reactive_protein'	-0.0402	0.0244	-1.65	0.0987
age	0.0071	0.0077	0.93	0.3542
age'	-0.0102	0.0094	-1.08	0.2781

Simulations function

Simulate baseline data

```
N <- 200000
baseline_data <- d |>
  filter(week == 1) |>
  select(yprev, ypprev, ecog_fstcnt, gender, diagnosis, age, albumin,
c_reactive_protein) |>
  slice_sample(n = N, replace = TRUE) |>
  mutate(
    pat_id = row_number(),
    tx = sample(c(0, 1), size = N, replace = TRUE),
    x = rnorm(N) # add patient specific offset
  ) |>
  data.frame()
```

Predict next states

```
# Vectorized version: computes category probabilities for all rows in `newdata` at once
# and returns an integer vector of sampled categories (1..k+1).
predict_next_state_vec <- function(model, newdata, k,
                                   tx_effect = log(1), x_effect = log(1)) {

  N <- nrow(newdata)

  # Get cumulative-logit linear predictors for each cutpoint, for ALL rows at once.
  # Had simple loop before, but vapply is much faster
  # Each call returns a length-N numeric vector.
  eta_mat <- vapply(
    X = seq_len(k),
    FUN = function(i) {
      as.numeric(predict(model, newdata = newdata, type = "lp", kint = i))
    },
    FUN.VALUE = numeric(N)
  )
  # Now eta_mat is N x K (rows: patients, cols: cutpoints 1..K)

  # Add external shifts (same shift to all cutpoints per row)
  shift <- newdata$tx * tx_effect + newdata$x * x_effect
  eta_mat <- sweep(eta_mat, 1, shift, FUN = "+")

  # Add +/- Inf boundaries, convert to probs via logistic, then difference adjacent
  CDFs
  # to get category probabilities (N x (k+1))
  eta_ext <- cbind(Inf, eta_mat, -Inf) # N x (K+2)
  p_ext <- plogis(eta_ext)
  sop <- p_ext[, 1:(k+1), drop = FALSE] - p_ext[, 2:(k+2), drop = FALSE] # N x
  (K+1)

  # Row-wise categorical sampling without loops: inverse-CDF method
  u <- runif(N) #between 0 and 1 by default
  # cumulative probabilities by row
  cs <- t(apply(sop, 1, cumsum)) # N x (K+1)
  # first index where cumprob >= u
  picks <- 1L + rowSums(cs < u) # integer in 1..(k+1)
```

```

    return(picks)
}

```

```

generate_trajectories <- function(model, baseline_data, times, k,
                                absorbing_state = NULL,
                                tx_effect = log(1), x_effect = log(1),
                                seed = 6789543, verbose = FALSE) {

  set.seed(seed)

  # Keep factors as factors; I only update their character levels once where needed.
  N <- nrow(baseline_data)

  # Validate factor coding just once
  yprev_levels <- levels(baseline_data$yprev)
  ypprev_levels <- levels(baseline_data$ypprev)
  if (is.null(yprev_levels) || is.null(ypprev_levels)) {
    stop("Error: 'yprev' and 'ypprev' must be factor variables.")
  }

  current <- baseline_data

  # Preallocate results list
  out <- vector("list", length(times))

  msg <- function(...) if (isTRUE(verbose)) message(sprintf(...))

  for (i in seq_along(times)) {
    t <- times[i]
    if (verbose) msg("Week %s (%d/%d)", t, i, length(times))

    # Set the week column once for all rows this step
    current$week <- t

    # Absorption mask: absorbed if previous state equals absorbing_state
    if (!is.null(absorbing_state)) {
      absorbed <- current$yprev == as.character(absorbing_state)
    } else {
      absorbed <- rep(FALSE, N)
    }

    # Sample next states for the non-absorbed patients in one vectorized call
    y_new <- integer(N)

    if (any(!absorbed)) {
      picks <- predict_next_state_vec(
        model, current[!absorbed, , drop = FALSE], k = k,
        tx_effect = tx_effect, x_effect = x_effect
      )
      y_new[!absorbed] <- picks
    }
    if (any(absorbed)) {
      y_new[absorbed] <- absorbing_state
    }
  }
}

```

```

# Store results for this week (keep as integers; convert to factor later if needed)
out[[i]] <- data.frame(
  pat_id = current$pat_id,
  week   = t,
  y      = y_new,
  stringsAsFactors = FALSE
)

# Roll the state forward:
# ypprev <- old yprev ; yprev <- new y
# (keep them as character to avoid refactor overhead inside the loop)
current$yprev <- as.character(current$yprev)
current$yprev <- as.character(y_new)
}

res <- do.call(rbind, out)
res$y <- factor(res$y, levels = seq_len(k + 1), ordered = TRUE)

res
}

```

h0: No treatment effect

```

patient_trajectories <- generate_trajectories(
  model = fit_h0,
  baseline_data = baseline_data,
  times = 1:26,
  k = 4,
  absorbing_state = 5,
  tx_effect = log(1),
  x_effect = log(1),
  seed = 6789543,
  verbose = FALSE
)

h0_data_forward <- baseline_data |>
  rename(
    yprev_base = yprev,
    ypprev_base = ypprev,
  ) |>
  left_join(patient_trajectories, by = "pat_id") |>
  arrange(pat_id, week) |>
  group_by(pat_id) |>
  mutate(
    yprev = lag(y, 1),
    ypprev = lag(y, 2),
    # For the first one or two weeks, seed lags from baseline
    yprev = ifelse(is.na(yprev), as.integer(as.character(yprev_base)),
  as.integer(as.character(yprev))),
    ypprev = ifelse(is.na(ypprev), as.integer(as.character(ypprev_base)),
  as.integer(as.character(ypprev))),
  ) |>
  mutate(dweek = ifelse(any(y == 5), min(week[y == 5]), max(week))) |>

```

```

ungroup()

h0_data <- h0_data_forward |>
  filter(week <= dweek) |>
  ungroup() |>
  mutate(
    y = as.numeric(y),
    yprev = factor(yprev, levels = 1:4, ordered = FALSE),
    ypprev = factor(ypprev, levels = 1:4, ordered = FALSE)
  ) |>
  select(-yprev_base, -ypprev_base, -dweek, -x)

fit_test <- orm(
  y ~ rcs(week, 5) + yprev + ypprev + week %ia% yprev + week %ia% ypprev +
  ecog_fstcnt + gender + diagnosis + rcs(albumin, 3) + rcs(c_reactive_protein, 3) +
  rcs(age, 3),
  data = h0_data
)

# we have very close agreement between model used to generate data and same model fit
# to generated data
# if we set the treatment effect and x effect to log(1)
cbind(round(coef(fit_h0), 3), round(coef(fit_test), 3))

```

	[,1]	[,2]
y>=2	-3.143	-3.155
y>=3	-3.885	-3.897
y>=4	-4.008	-4.020
y>=5	-6.960	-6.977
week	0.033	0.034
week'	-0.581	-0.595
week''	1.479	1.524
week'''	-1.625	-1.691
yprev=2	1.625	1.623
yprev=3	0.547	0.537
yprev=4	3.468	3.477
ypprev=2	0.067	0.050
ypprev=3	2.140	2.200
ypprev=4	-1.112	-1.110
week * yprev=2	0.064	0.063
week * yprev=3	0.008	0.007
week * yprev=4	0.049	0.048
week * ypprev=2	-0.008	-0.008
week * ypprev=3	-0.193	-0.200
week * ypprev=4	0.033	0.033
ecog_fstcnt=2	0.360	0.366
ecog_fstcnt=3	0.506	0.526
gender=1	0.225	0.231
diagnosis=2	-0.153	-0.154
diagnosis=3	-0.288	-0.289
diagnosis=4	-1.056	-1.069
diagnosis=5	0.016	0.010
albumin	0.000	0.000

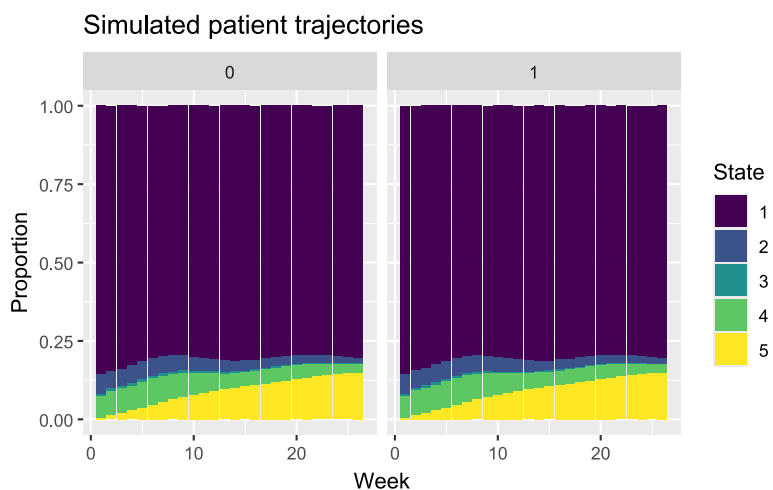
albumin'	0.021	0.020
c_reactive_protein	0.015	0.014
c_reactive_protein'	-0.040	-0.042
age	0.007	0.007
age'	-0.010	-0.012

export

```
write_parquet(h0_data, here("2-hosp", "sim-data", "h0.parquet"))
write_parquet(h0_data_forward, here("2-hosp", "sim-data", "h0_forward.parquet"))
```

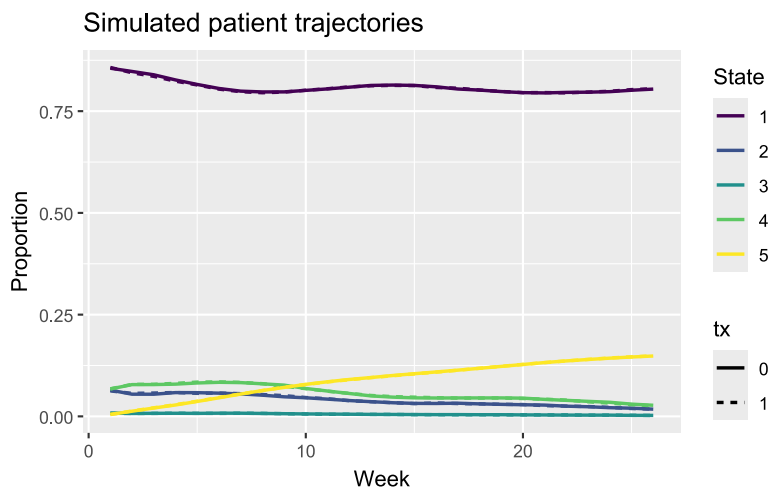
Visualize simulated data

```
baseline_data |>
  left_join(patient_trajectories, by = "pat_id") |>
  group_by(tx, week, y) |>
  summarise(n = n(), .groups = "drop") |>
  group_by(tx, week) |>
  mutate(prop = n / sum(n)) |>
  ggplot(aes(x = week, y = prop)) +
  # stacked bar char
  geom_bar(aes(fill = factor(y)), stat = "identity", position = "fill") +
  facet_wrap(~ tx) +
  labs(
    title = "Simulated patient trajectories",
    x = "Week",
    y = "Proportion",
    fill = "State",
    color = "State"
  )
```



```
baseline_data |>
  left_join(patient_trajectories, by = "pat_id") |>
  group_by(tx, week, y) |>
  summarise(n = n(), .groups = "drop") |>
  group_by(tx, week) |>
  mutate(prop = n / sum(n)) |>
```

```
mutate(tx = factor(tx)) |>
ggplot(aes(x = week, y = prop, color = factor(y), linetype = tx)) +
geom_line(linewidth = 0.8) +
labs(
  title = "Simulated patient trajectories",
  x = "Week",
  y = "Proportion",
  fill = "State",
  color = "State"
)
```



HA: Treatment effect

Or of 0.9 ~ 4 days more at home and alive

```
patient_trajectories_09 <- generate_trajectories(
  model = fit_h0,
  baseline_data = baseline_data,
  times = 1:26,
  k = 4,
  absorbing_state = 5,
  tx_effect = log(0.9),
  x_effect = log(1),
  seed = 6789543,
  verbose = FALSE
)

ha_data_forward_09 <- baseline_data |>
  rename(
    yprev_base = yprev,
    ypprev_base = ypprev,
  ) |>
  left_join(patient_trajectories_09, by = "pat_id") |>
  arrange(pat_id, week) |>
  group_by(pat_id) |>
  mutate(
    yprev = lag(y, 1),
    ypprev = lag(y, 2),
  )
```



```

# For the first one or two weeks, seed lags from baseline
yprev = ifelse(is.na(yprev), as.integer(as.character(yprev_base)),
as.integer(as.character(yprev))),
ypprev = ifelse(is.na(ypprev), as.integer(as.character(ypprev_base)),
as.integer(as.character(ypprev)))
) |>
mutate(dweek = ifelse(any(y == 5), min(week[y == 5]), max(week)))

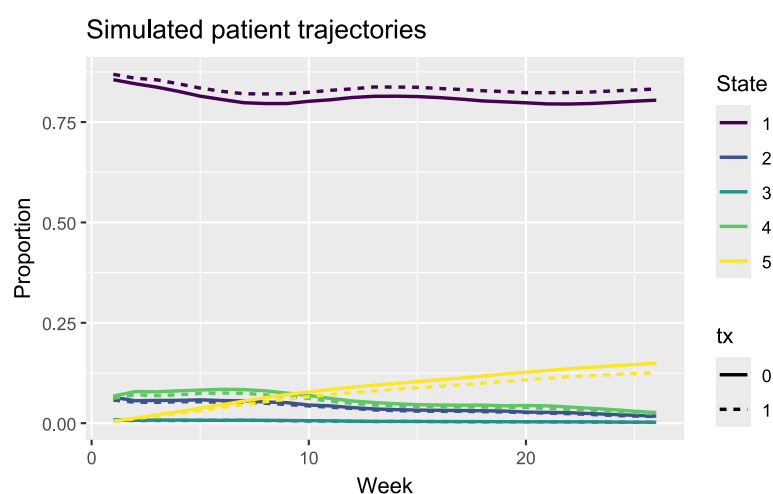
ha_data_09 <- ha_data_forward_09 |>
filter(week <= dweek) |>
ungroup() |>
mutate(
  y = as.numeric(y),
  yprev = factor(yprev, levels = 1:4, ordered = FALSE),
  ypprev = factor(ypprev, levels = 1:4, ordered = FALSE)
) |>
select(-yprev_base, -ypprev_base, -dweek, -x)

```

```

baseline_data |>
left_join(patient_trajectories_09, by = "pat_id") |>
group_by(tx, week, y) |>
summarise(n = n(), .groups = "drop") |>
group_by(tx, week) |>
mutate(prop = n / sum(n)) |>
mutate(tx = factor(tx)) |>
ggplot(aes(x = week, y = prop, color = factor(y), linetype = tx)) +
geom_line(linewidth = 0.8) +
labs(
  title = "Simulated patient trajectories",
  x = "Week",
  y = "Proportion",
  fill = "State",
  color = "State"
)

```



```

sops <- baseline_data |>
left_join(patient_trajectories_09, by = "pat_id") |>

```

```

group_by(tx, week, y) |>
summarise(n = n(), .groups = "drop") |>
group_by(tx, week) |>
mutate(prop = n / sum(n)) |>
mutate(tx = factor(tx))

# difference in time out of hospital
sops |>
  group_by(tx) |>
  filter(y == 1) |>
  summarise(mean_time = sum(prop))

```

```

# A tibble: 2 × 2
  tx    mean_time
<fct>    <dbl>
1 0         21.0
2 1         21.7

```

OR of 0.8 ~ 1.2 more weeks at home and alive

```

patient_trajectories_08 <- generate_trajectories(
  model = fit_h0,
  baseline_data = baseline_data,
  times = 1:26,
  k = 4,
  absorbing_state = 5,
  tx_effect = log(0.8),
  x_effect = log(1),
  seed = 6789543,
  verbose = FALSE
)

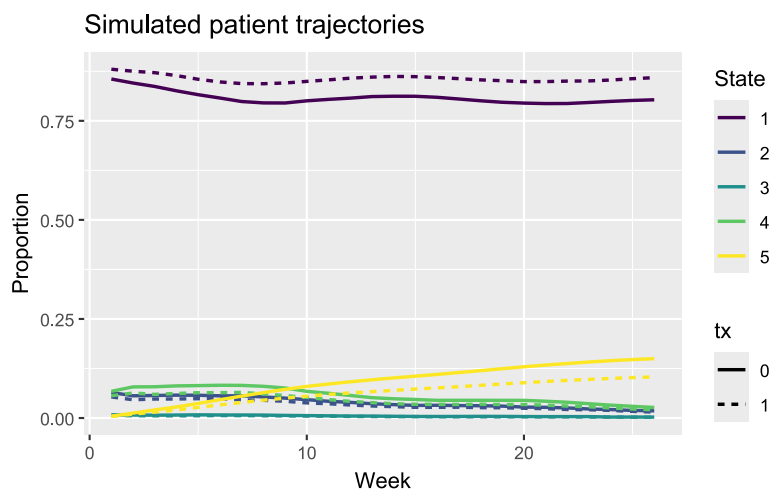
ha_data_forward_08 <- baseline_data |>
  rename(
    yprev_base = yprev,
    ypprev_base = ypprev,
  ) |>
  left_join(patient_trajectories_08, by = "pat_id") |>
  arrange(pat_id, week) |>
  group_by(pat_id) |>
  mutate(
    yprev = lag(y, 1),
    ypprev = lag(y, 2),
    # For the first one or two weeks, seed lags from baseline
    yprev = ifelse(is.na(yprev), as.integer(as.character(yprev_base)),
as.integer(as.character(yprev))),
    ypprev = ifelse(is.na(ypprev), as.integer(as.character(ypprev_base)),
as.integer(as.character(ypprev)))
  ) |>
  mutate(dweek = ifelse(any(y == 5), min(week[y == 5]), max(week)))

ha_data_08 <- ha_data_forward_08 |>
  filter(week <= dweek) |>
  ungroup() |>

```

```
mutate(
  y = as.numeric(y),
  yprev = factor(yprev, levels = 1:4, ordered = FALSE),
  ypprev = factor(ypprev, levels = 1:4, ordered = FALSE)
) |>
select(-yprev_base, -ypprev_base, -dweek, -x)
```

```
baseline_data |>
  left_join(patient_trajectories_08, by = "pat_id") |>
  group_by(tx, week, y) |>
  summarise(n = n(), .groups = "drop") |>
  group_by(tx, week) |>
  mutate(prop = n / sum(n)) |>
  mutate(tx = factor(tx)) |>
  ggplot(aes(x = week, y = prop, color = factor(y), linetype = tx)) +
  geom_line(linewidth = 0.8) +
  labs(
    title = "Simulated patient trajectories",
    x = "Week",
    y = "Proportion",
    fill = "State",
    color = "State"
  )
)
```



```
sops <- baseline_data |>
  left_join(patient_trajectories_08, by = "pat_id") |>
  group_by(tx, week, y) |>
  summarise(n = n(), .groups = "drop") |>
  group_by(tx, week) |>
  mutate(prop = n / sum(n)) |>
  mutate(tx = factor(tx))

# difference in time out of hospital
sops |>
  group_by(tx) |>
  filter(y == 1) |>
  summarise(mean_time = sum(prop))
```

```
# A tibble: 2 × 2
  tx    mean_time
<fct>    <dbl>
1 0         21.0
2 1         22.3
```

OR of 0.7 ~ 1.8 more weeks at home and alive

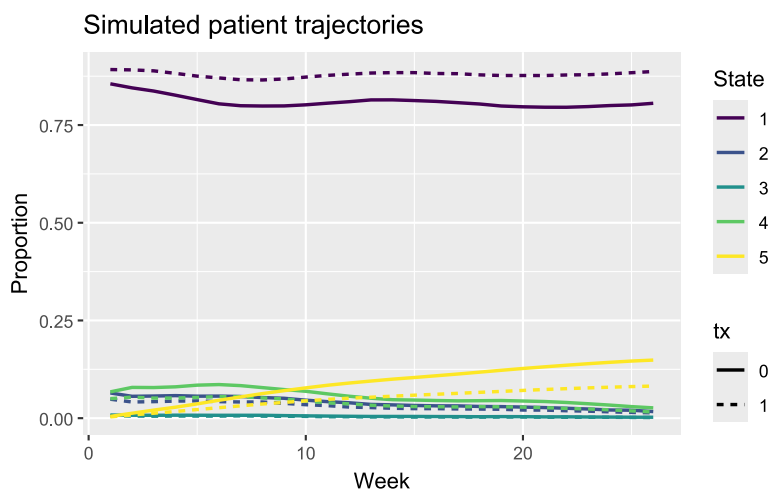
```
patient_trajectories_07 <- generate_trajectories(
  model = fit_h0,
  baseline_data = baseline_data,
  times = 1:26,
  k = 4,
  absorbing_state = 5,
  tx_effect = log(0.7),
  x_effect = log(1),
  seed = 6789543,
  verbose = FALSE
)

ha_data_forward_07 <- baseline_data |>
  rename(
    yprev_base = yprev,
    ypprev_base = ypprev,
  ) |>
  left_join(patient_trajectories_07, by = "pat_id") |>
  arrange(pat_id, week) |>
  group_by(pat_id) |>
  mutate(
    yprev = lag(y, 1),
    ypprev = lag(y, 2),
    # For the first one or two weeks, seed lags from baseline
    yprev = ifelse(is.na(yprev), as.integer(as.character(yprev_base)),
as.integer(as.character(yprev))),
    ypprev = ifelse(is.na(ypprev), as.integer(as.character(ypprev_base)),
as.integer(as.character(ypprev)))
  ) |>
  mutate(dweek = ifelse(any(y == 5), min(week[y == 5]), max(week)))

ha_data_07 <- ha_data_forward_07 |>
  filter(week <= dweek) |>
  ungroup() |>
  mutate(
    y = as.numeric(y),
    yprev = factor(yprev, levels = 1:4, ordered = FALSE),
    ypprev = factor(ypprev, levels = 1:4, ordered = FALSE)
  ) |>
  select(-yprev_base, -ypprev_base, -dweek, -x)
```

```
baseline_data |>
  left_join(patient_trajectories_07, by = "pat_id") |>
  group_by(tx, week, y) |>
  summarise(n = n(), .groups = "drop") |>
  group_by(tx, week) |>
```

```
mutate(prop = n / sum(n)) |>
mutate(tx = factor(tx)) |>
ggplot(aes(x = week, y = prop, color = factor(y), linetype = tx)) +
geom_line(linewidth = 0.8) +
labs(
  title = "Simulated patient trajectories",
  x = "Week",
  y = "Proportion",
  fill = "State",
  color = "State"
)
```



```
sops <- baseline_data |>
left_join(patient_trajectories_07, by = "pat_id") |>
group_by(tx, week, y) |>
summarise(n = n(), .groups = "drop") |>
group_by(tx, week) |>
mutate(prop = n / sum(n)) |>
mutate(tx = factor(tx))

# difference in time out of hospital
sops |>
group_by(tx) |>
filter(y == 1) |>
summarise(mean_time = sum(prop))
```

```
# A tibble: 2 × 2
  tx    mean_time
<fct>    <dbl>
1 0         21.1
2 1         22.9
```

Export

```
write_parquet(ha_data_forward_07, here("2-hosp", "sim-data", "ha_07_forward.parquet"))
write_parquet(ha_data_07, here("2-hosp", "sim-data", "ha_07.parquet"))
write_parquet(ha_data_forward_08, here("2-hosp", "sim-data", "ha_08_forward.parquet"))
```

```
write_parquet(ha_data_08, here("2-hosp", "sim-data", "ha_08.parquet"))  
write_parquet(ha_data_forward_09, here("2-hosp", "sim-data", "ha_09_forward.parquet"))  
write_parquet(ha_data_09, here("2-hosp", "sim-data", "ha_09.parquet"))
```